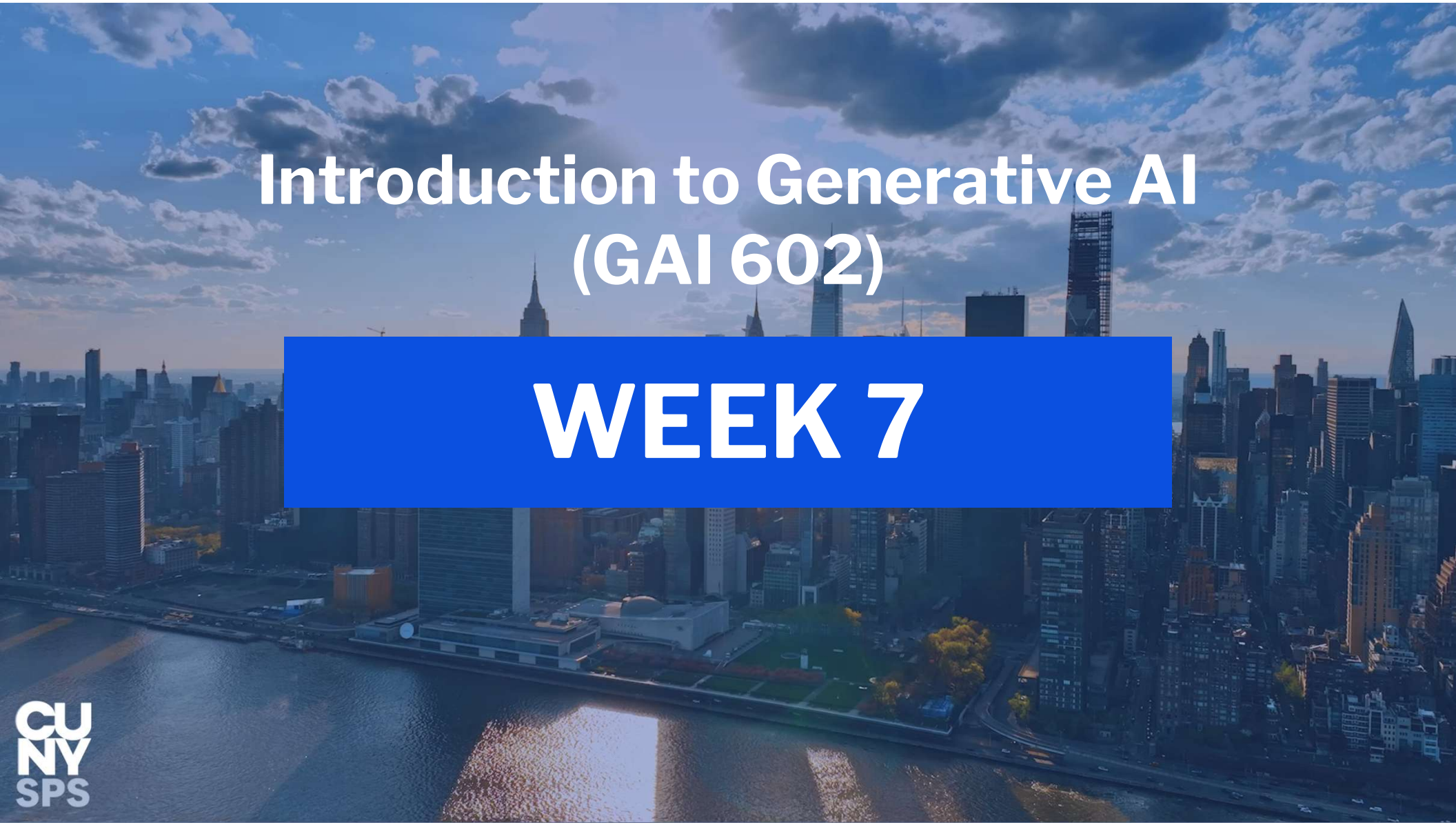

Introduction to Generative AI
(GAI 602)
WEEK 7
CU
NY
SPS



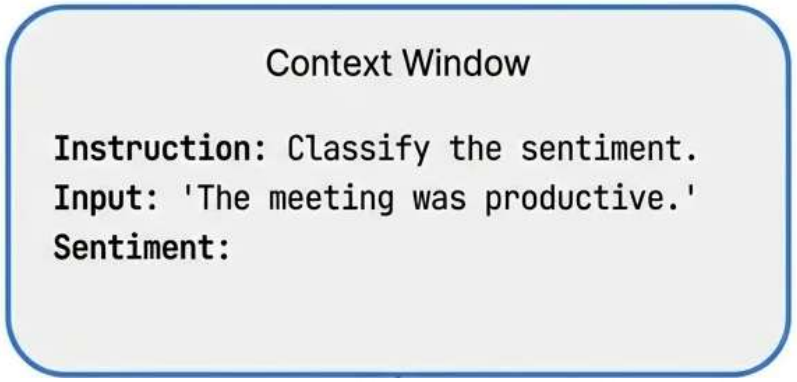
Prompting

Zero-shot

Zero-shot prompting involves providing the model with a direct instruction or question without any additional context or examples.

Examples of this are:

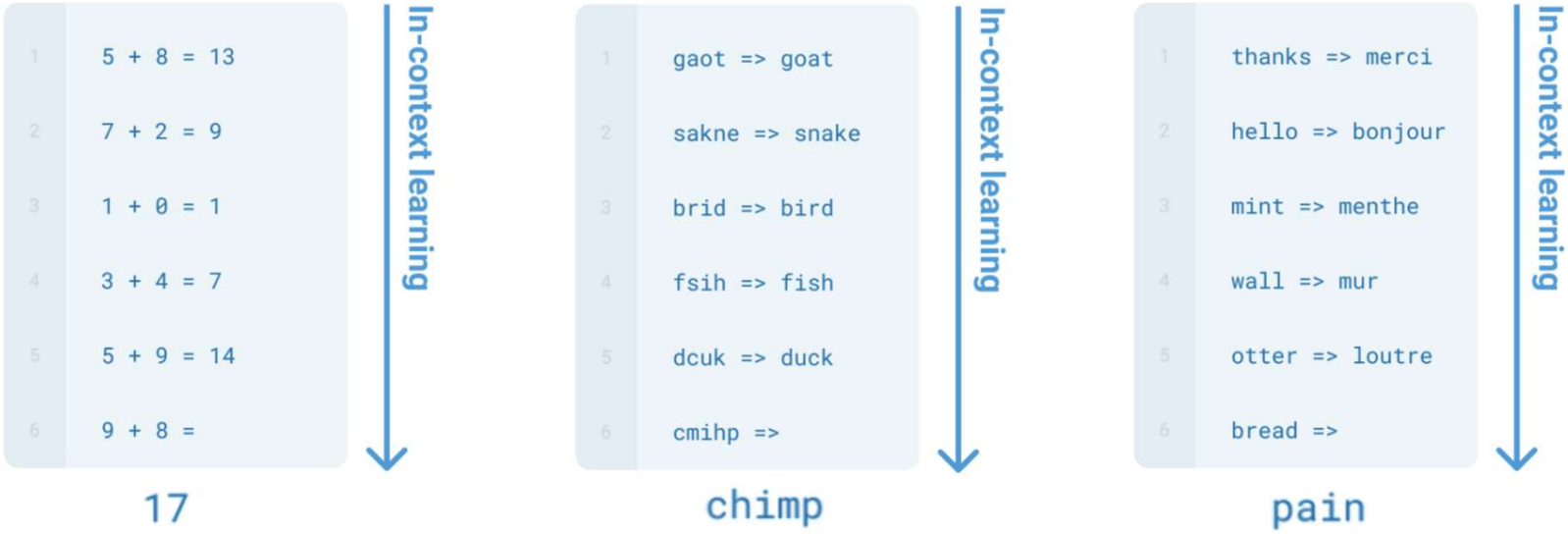
- Idea generation, where the model is prompted to generate creative ideas or brainstorming solutions
- Summarization, or translation, where the model is asked to summarize or translate some piece of content



Source: Google

Few-shot

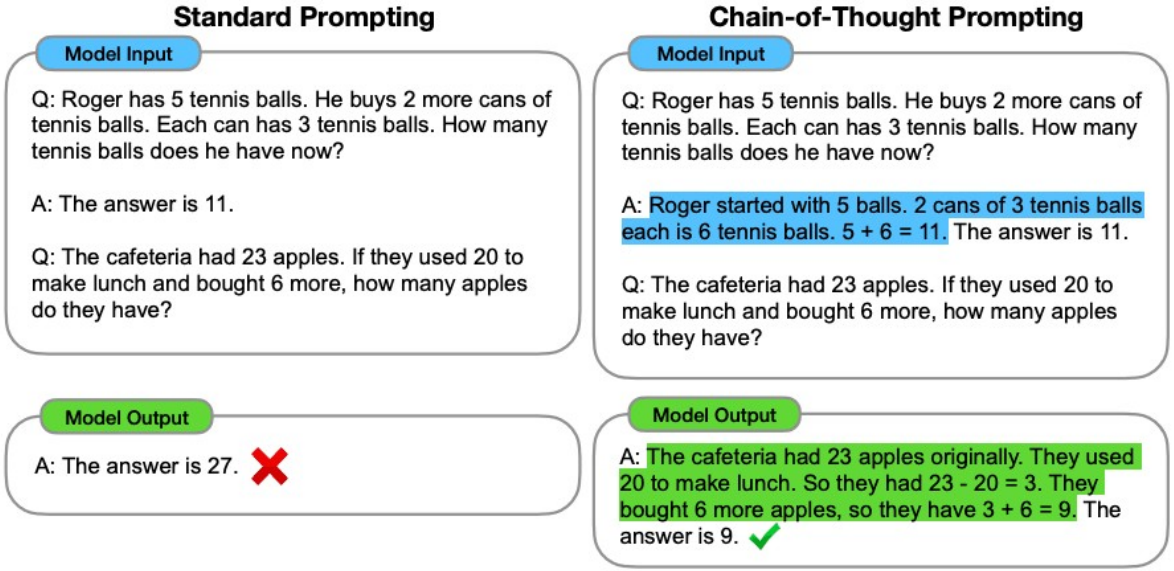
- Provide one, two or more examples of complete questions and answers
- This can help the model better “understand” the task and generate more accurate responses.
- The system learns the style of the examples
- When you give it a new question, it gives an answer that follows the same structure



Source: Paper: “Language Models are Few Short Learners”, https://arxiv.org/abs/2005.14165

Chain-of-Thought (CoT)

- Asking the model to break down complex reasoning into a series of intermediate steps, leading to a more comprehensive and well-structured final output.
- The system breaks down the rationale, and shows the user how the steps were organized, and helps identify issues
- Good for mathematics and logical reasoning



Source: Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in LLMs" - https://arxiv.org/abs/2201.11903

Self-criticism

Zero-Shot-CoT

Q: Valerie earns $5000 per month, 1/2 of what her brother earns. If their mother earns twice their combined salary, what's the total amount of money they all have together?
A: Let's think step by step. First, we need to figure out how much Valerie's brother earns. We know that Valerie earns 1/2 of what her brother earns, so we can set up an equation:

⋮

Next, we need to find out how much their mother earns. We know that their mother earns twice their combined salary, so we can add up Valerie and Brother's earnings and multiply by 2: Total earnings = 2 * (Valerie's earnings + Brother's earnings) Total earnings = 2 * ($5000 + $10,000) Total earnings = $30,000. Therefore, the total amount of money they all have together is $30,000. ✗

RCI

If Valerie earns 1/2 of what her brother earns, then her brother earns 2*$5000 = $<<2*5000=10000>>10000 per month.

⋮

Therefore, the total amount of money they all have together is $30000.

Review your previous answer and find problems with your answer.

All individuals, including Valerie, her brother, and mother, should be taken into account when calculating the overall sum of money.

Based on the problems you found, improve your answer.

I apologize for the mistake in my previous answer. Here's the correct solution: If Valerie earns 1/2 of what her brother earns, then her brother earns 2*$5000 = $<<2*5000=10000>>10000 per month. Together, Valerie and her brother earn $5000+$10000 = $<<5000+10000=15000>>15000 per month. If their mother earns twice their combined salary, then she earns 2*$15000 = $<<2*15000=30000>>30000 per month. Therefore, the total amount of money they all have together is $5000+$10000+$30000 = $<<5000+10000+30000=45000>>45000. ✓

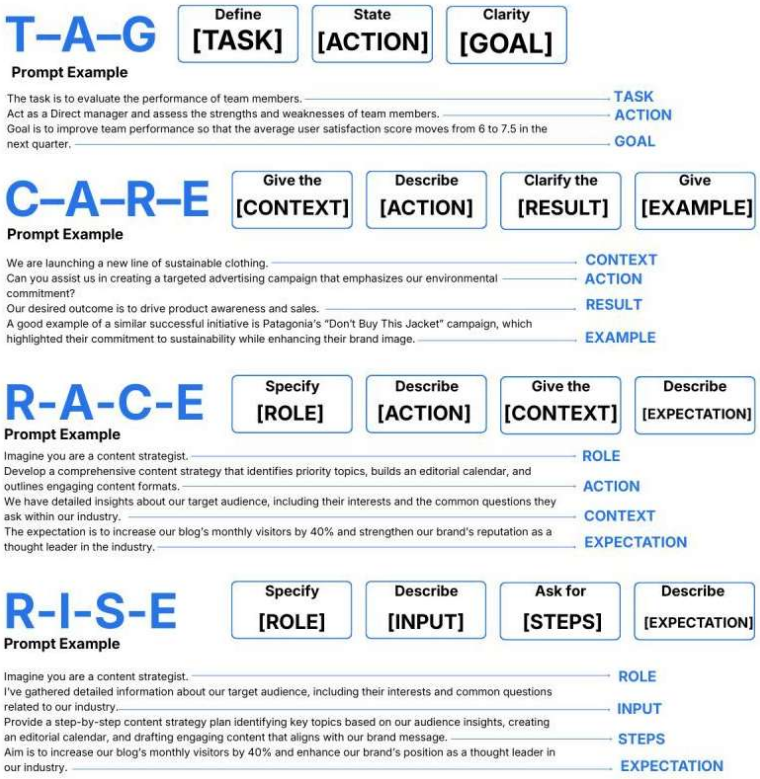
Source: Kim et al., “LMs can Solve Computer Tasks” - https://arxiv.org/abs/2303.17491

Best Practices

Combine (ensemble) the techniques

1. Put the instructions at the beginning of the prompt
2. Use delimiters
3. Be very specific
4. Give the model a persona
5. Provide relevant examples
6. Ask the model to explain the chain of thought
7. Specify the desired output format
8. Supply the AI with relevant data
9. Ask for citations (to avoid hallucination)
10. Prompt in iterations

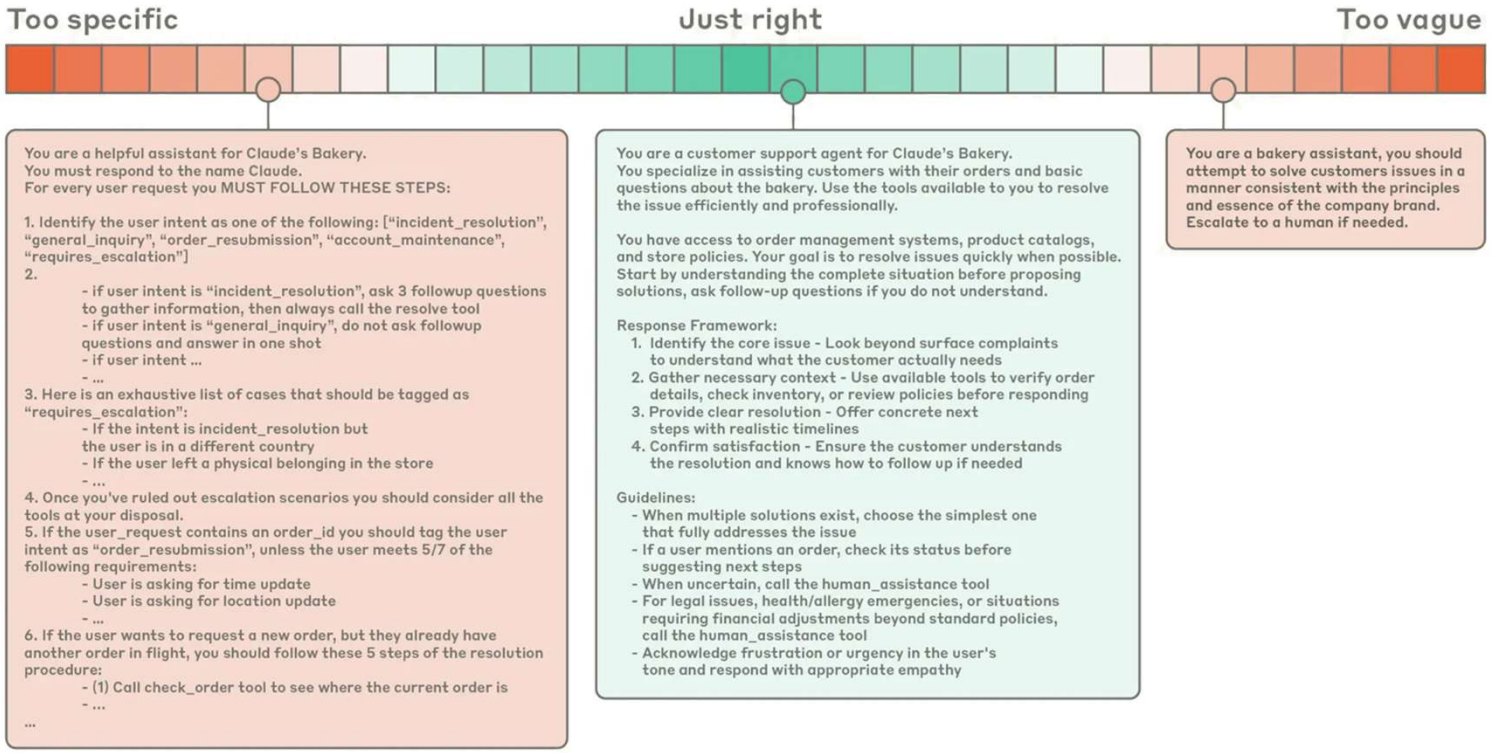
Use frameworks



and there are many more...

Source: hostinger

Calibrating the Prompt

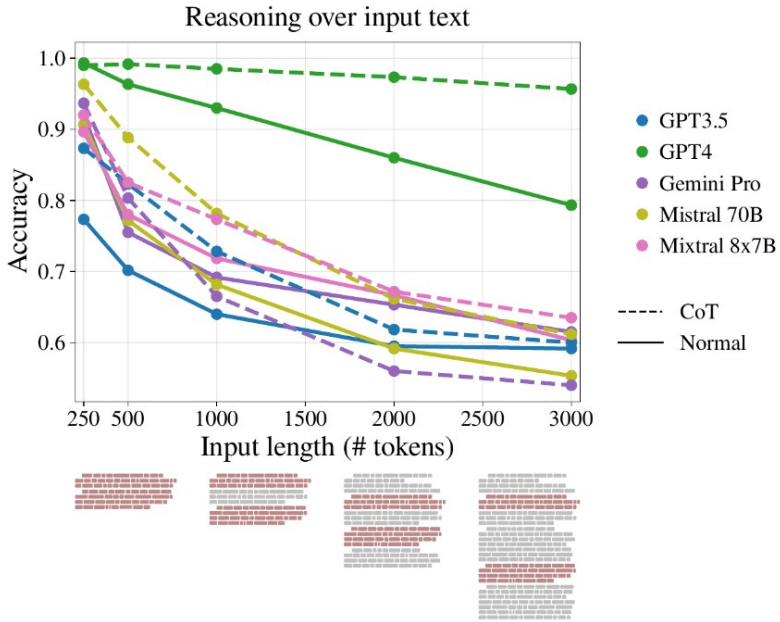


Source: Anthropic & epoch.ai

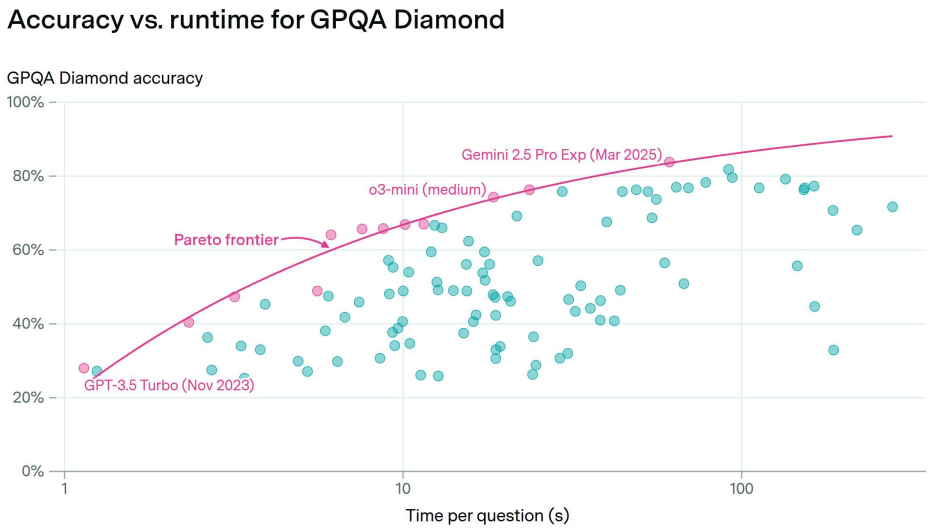
Prompt Complexity

- Longer prompts can overwhelm the model's processing capabilities, leading to a decline in performance.
- Longer prompts may require a longer time to process
- Shorter prompts may result in insufficient information and, hence, lower of performance
- Some models pay more attention to information at the beginning of the context (so order matters)

Longer prompts don't always do better



Longer prompts may be slower



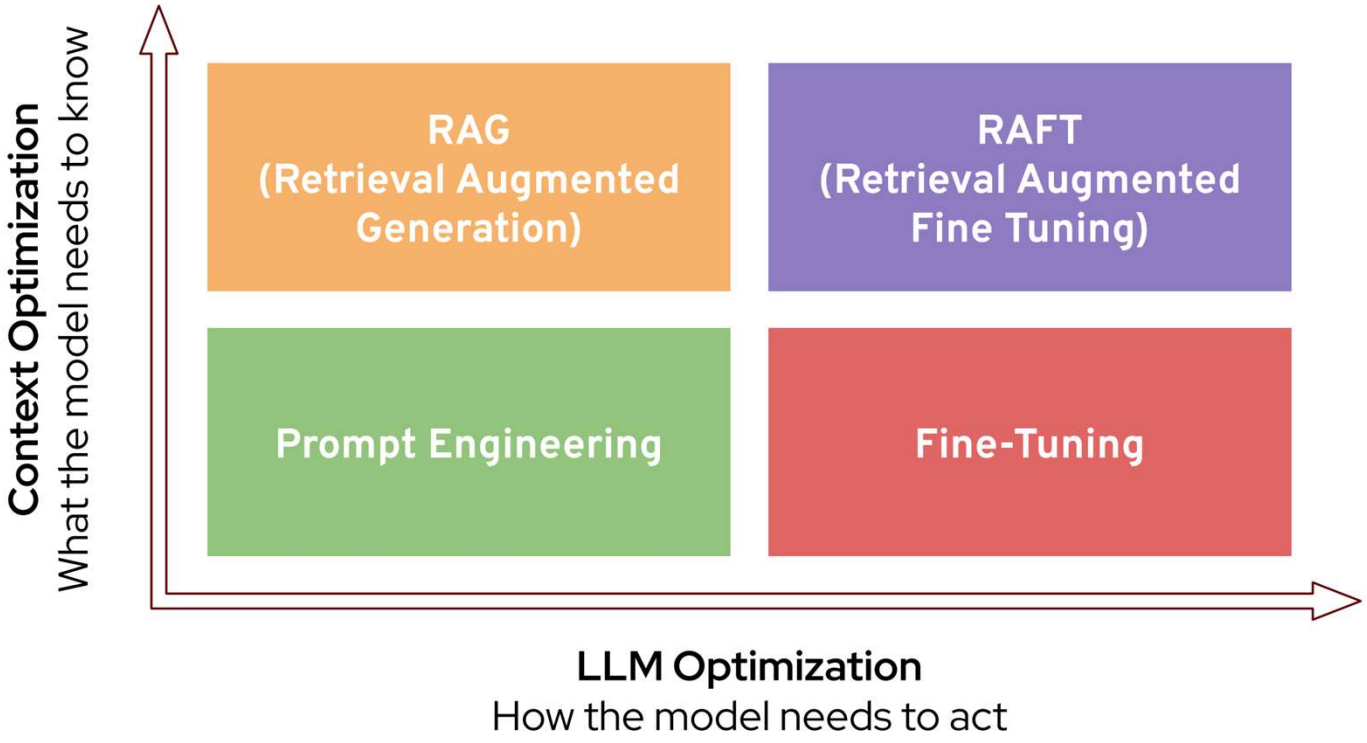
Sour



Context Engineering - Introduction

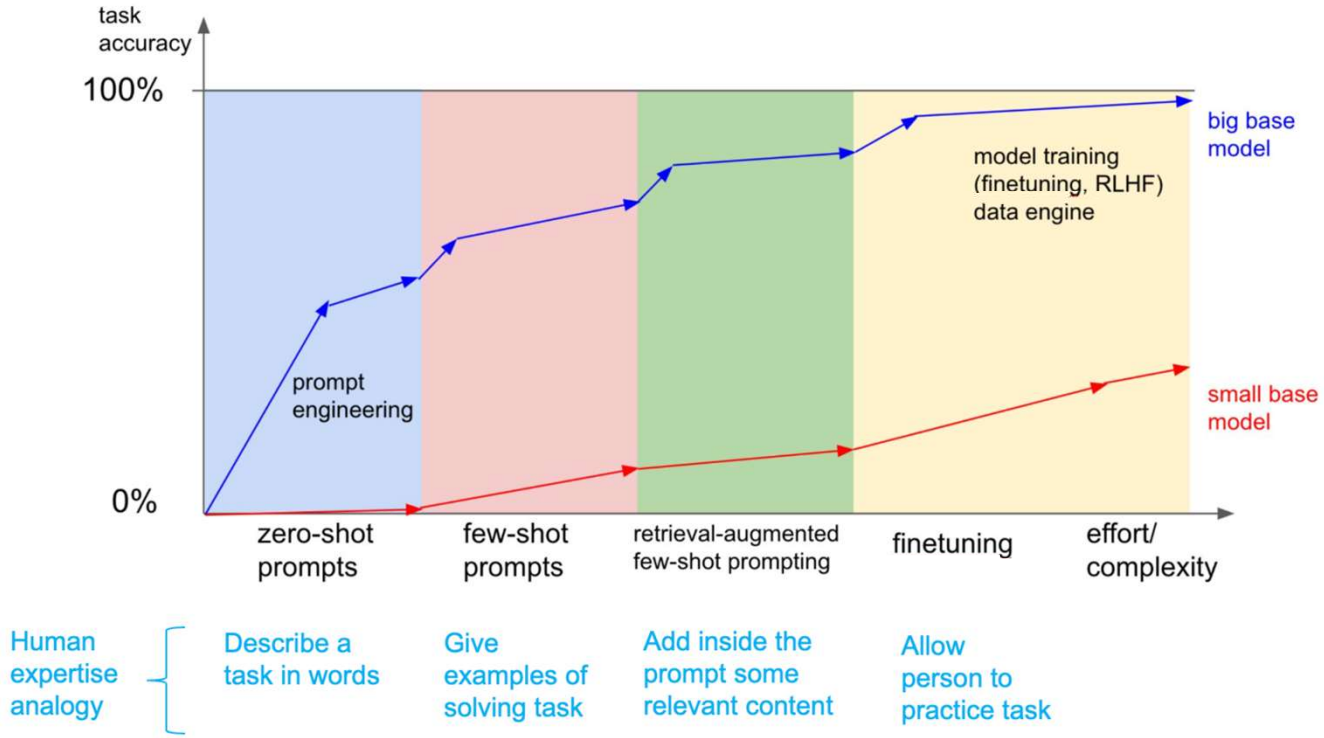
Optimizing the performance of LLMs

Improving the outputs of the model is an art as well as a science



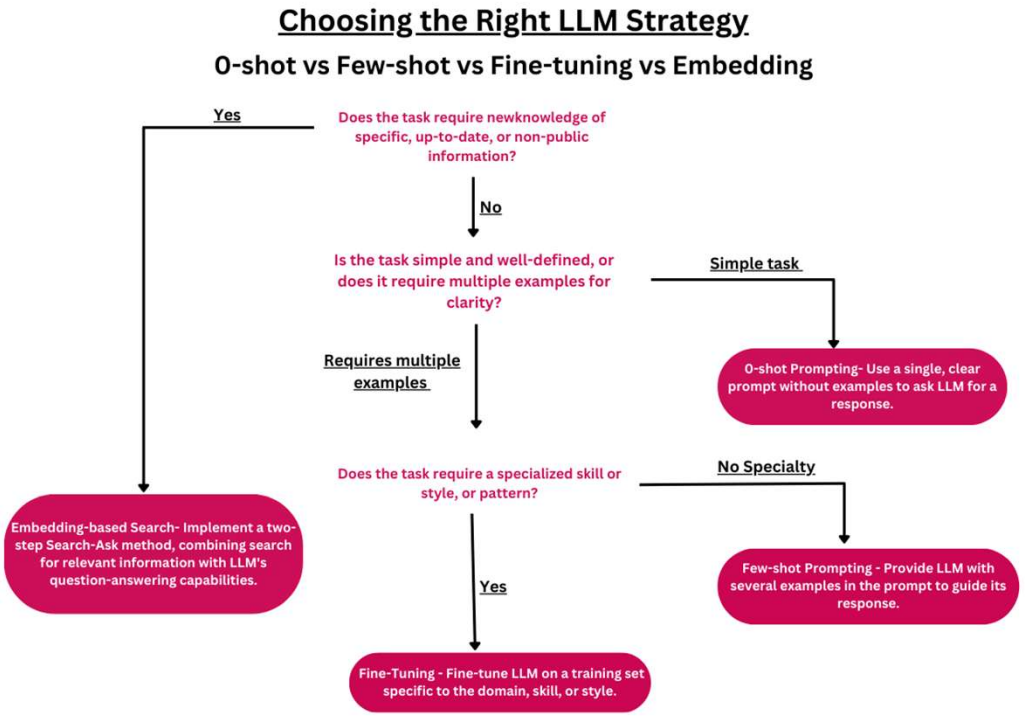
Source: Redhat

Optimizing the performance of LLMs



Source: Google

Optimizing the performance of LLMs



Source: promptengineering.org

“How to” Guide for Good Prompting Practices

1 Choose prompting technique

- Determine the most suitable prompting method—zero-shot, one-shot, or few-shot—based on the specific research task.

2 Enhance prompt clarity

- Structure the prompt with clearly defined sections to minimize ambiguity, facilitating more reliable and reproducible outcomes.

3 Provide context in prompt

- Background or contextual data to guide the model's understanding, enhancing the relevance and accuracy of responses.

4 Adjust exemplar complexity

- Select training examples with an appropriate level of complexity, ensuring alignment with the task’s cognitive and computational demands.

5 Highlight relevant information

- Clearly highlight key elements within the prompt to direct the model's focus, ensuring optimal processing of relevant details.

6 Iterative refinement

- Continuously optimize the prompt based on iterative analysis of model outputs to enhance performance and accuracy.

7 Hypertune parameters

- Adjust hyperparameters systematically to identify the optimal configuration that maximizes task-specific performance.

8 Report results

- Report results in a clear and structured manner, linking model performance to the chosen prompting technique.

Source: Analytics Vidhya



(Naïve) Prompt Engineering

Consistency Challenges in LLMs – Input/Output

Temperature isn't the only reason LLM outputs aren't predictable.

Hallucinations

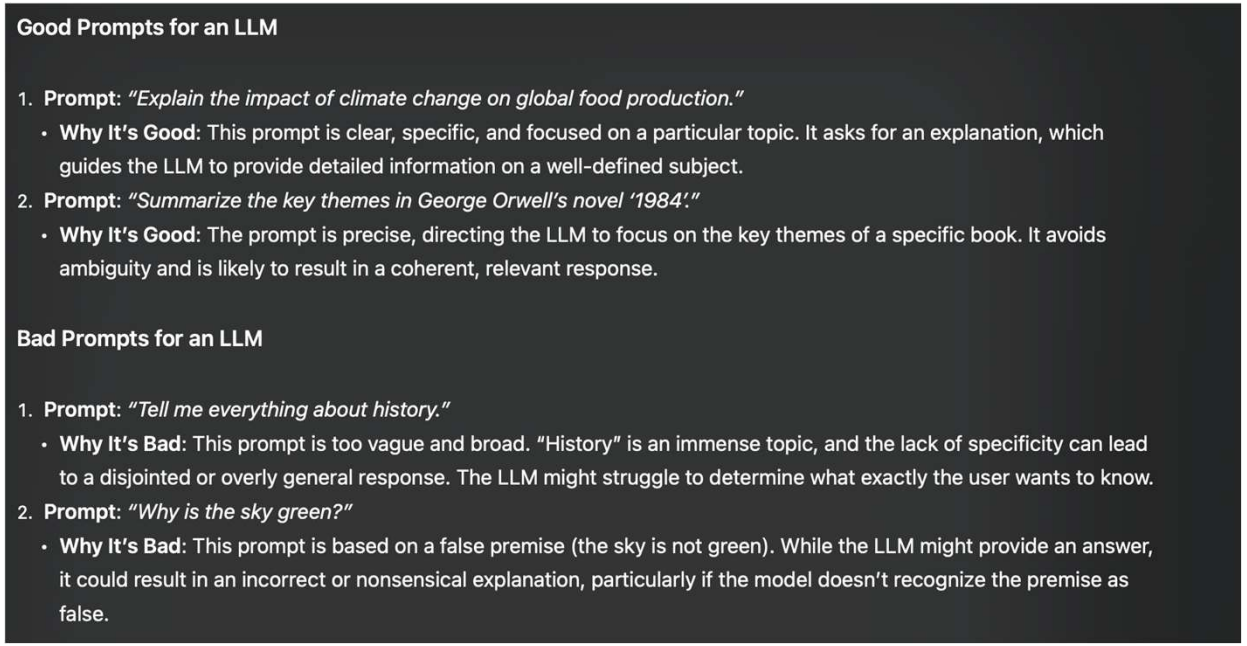
- An LLM has access to an astoundingly high dimensional space, constructed from the embedding dimensions, and number of parameters
- Only a tiny subset of this space will be accessed during training, with the number of potential input token configurations to the model effectively infinite
- This can lead LLMs, at inference time, to output data that may have no real connection to the input data, simply because it hasn't been well constrained
- This is known as hallucination. This occurs when the LLM attempts to extrapolate in a region of this high dimensional space where it has not been well trained.

Source: Nvidia

Prompting an LLM – How does it affect the model?

To over come hallucination, we can format our inputs in such a manner that we can generate consistency. Our prompts and prompt templates will be key.

By giving a consistent input to our models, we can promote a consistent activation of the right parameters and utilization of the attention mechanism



Source: Nvidia

What we've learned about prompting LLMs

Prompt templating is now a very common practice with multiple libraries now built to support simple templating

These templates typically align with various tasks:
- Summarization
- Analysis
- Translation
- Extraction

Each of which will have standardized templates that have been found to work best with various LLMs

```
prompt_template = PromptTemplate(
    input_variables=["theme"],  # List of input variable names
    template="Write a story about {theme}"  # String template with placeholders
)
```

```
summary_template = PromptTemplate.from_template(
    """Provide a concise summary of the following factual topic:

{topic}

Here are some key points to consider including:

* Background information
* Main ideas
* Supporting details
* Significance or impact

Strive for objectivity and neutrality in your summary."""
)

# Generate a summary prompt for climate change
climate_summary_prompt = summary_template.format(topic="Climate
Change")
print(climate_summary_prompt)
```

Source: Nvidia

Prompt Formats in Chat Models

Not all prompts are the same

In an API request, most Chat LLMs utilize three types of prompts/messages:
1. The System Message
2. The Assistant Response
3. The User Prompt

After each interaction, the messages array is expanded to contain the history of the assistant and user prompt, and the system message/prompt is pre-pended each time the user sends a message (though this isn't stored in the array)

```
client = OpenAI()

completion = client.chat.completions.create(
  model="gpt-4o",
  messages=[
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ]
)
```

Example API request format for OpenAI

Source: Nvidia

System Prompts

These special, static, prompts are pre-pended to the user prompts and the models are trained to obey these system prompts.

- **Guidance**: It sets the rules or guidelines for how the AI should respond to user inputs. This could include instructions on maintaining a certain tone (e.g., formal, casual), following specific ethical guidelines, or limiting responses to certain types of information.
- **Context**: It provides the AI with context about the interaction. For example, the system prompt might indicate that the AI should behave as a customer service representative or as a tutor in a specific subject.
- **Consistency**: It ensures that the AI's responses are consistent throughout the interaction, adhering to the predefined rules or context.

Example of a System Prompt

Let's say you're designing an AI assistant for a customer service chatbot. The system prompt might look something like this:

System Prompt: "*You are a helpful and polite customer service assistant. Always address customers respectfully, provide clear and concise answers, and offer additional help when possible. Avoid technical jargon and ensure the customer feels valued. If you cannot assist directly, offer to escalate the issue to a human representative.*"

```
You are ChatGPT, a large language model trained by OpenAI, based on the GPT-4 architecture.
Knowledge cutoff: 2023-10
Current date: 2024-08-12

Image input capabilities: Enabled
Personality: v2

# Tools

## bio

The `bio` tool allows you to persist information across conversations. Address your message `to=bio` and
write whatever information you want to remember. The information will appear in the model set context below
in future conversations.

## dalle

// Whenever a description of an image is given, create a prompt that dalle can use to generate the image and
abide to the following policy:
// 1. The prompt must be in English. Translate to English if needed.
// 2. DO NOT ask for permission to generate the image, just do it!
// 3. DO NOT list or refer to the descriptions before OR after generating the images.
// 4. Do not create more than 1 image, even if the user requests more.
// 5. Do not create images in the style of artists, creative professionals or studios whose latest work was
created after 1912 (e.g. Picasso, Kahlo).
// - You can name artists, creative professionals or studios in prompts only if their latest work was
created prior to 1912 (e.g. Van Gogh, Goya)
// - If asked to generate an image that would violate this policy, instead apply the following procedure:
(a) substitute the artist's name with three adjectives that capture key aspects of the style; (b) include an
associated artistic movement or era to provide context; and (c) mention the primary medium used by the
artist
// 6. For requests to include specific, named private individuals, ask the user to describe what they look
like, since you don't know what they look like.
// 7. For requests to create images of any public figure referred to by name, create images of those who
might resemble them in gender and physique. But they shouldn't look like them. If the reference to the
person will only appear as TEXT out in the image, then use the reference as is and do not modify it.
// 8. Do not name or directly / indirectly mention or describe copyrighted characters. Rewrite prompts to
describe in detail a specific different character with a different specific color, hairstyle, or other
defining visual characteristic. Do not discuss copyright policies in responses.
// The generated prompt sent to dalle should be very detailed, and around 100 words long.
// Example dalle invocation:
// ```
// {
// "prompt": "<insert prompt here>"
// }
// ```
```

Source: Nvidia


CUNY | School of
Professional Studies